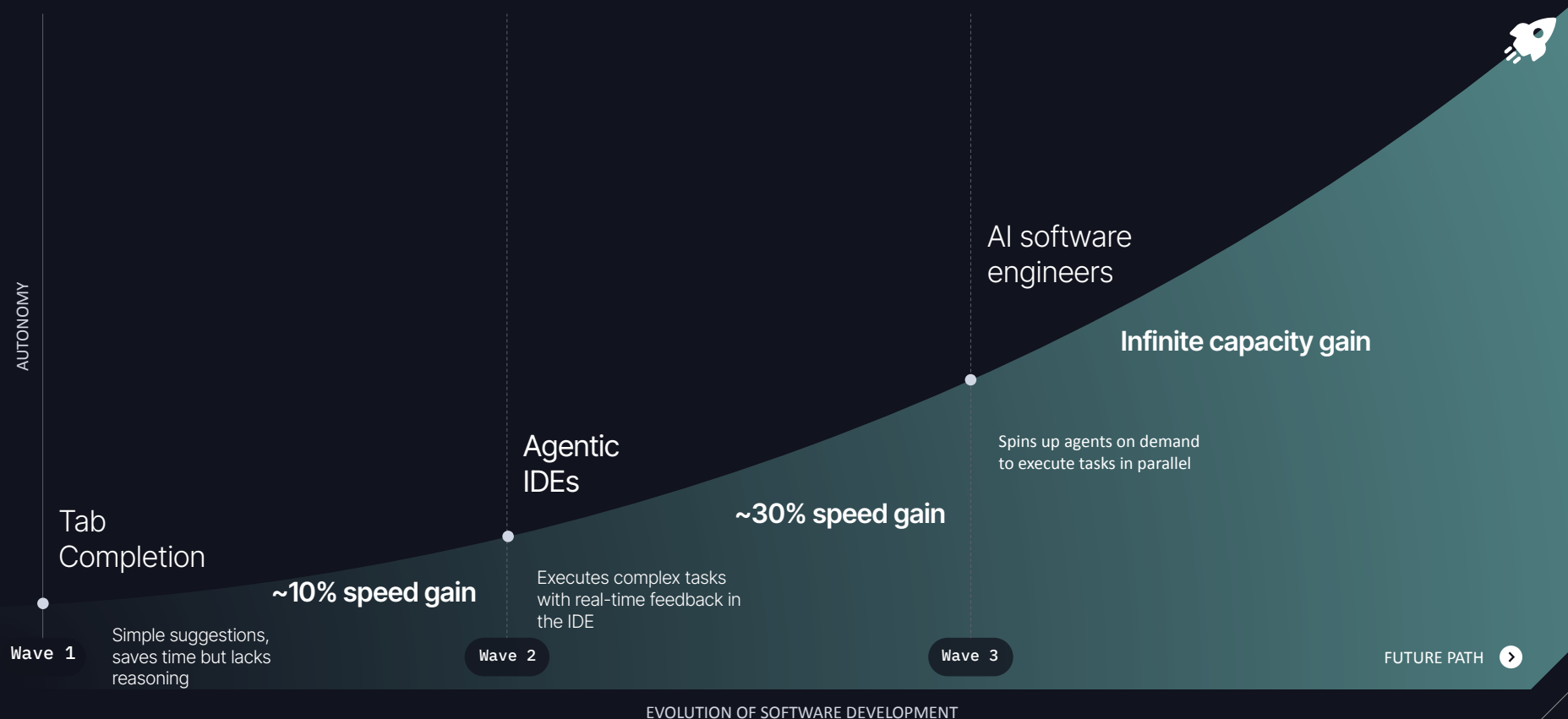




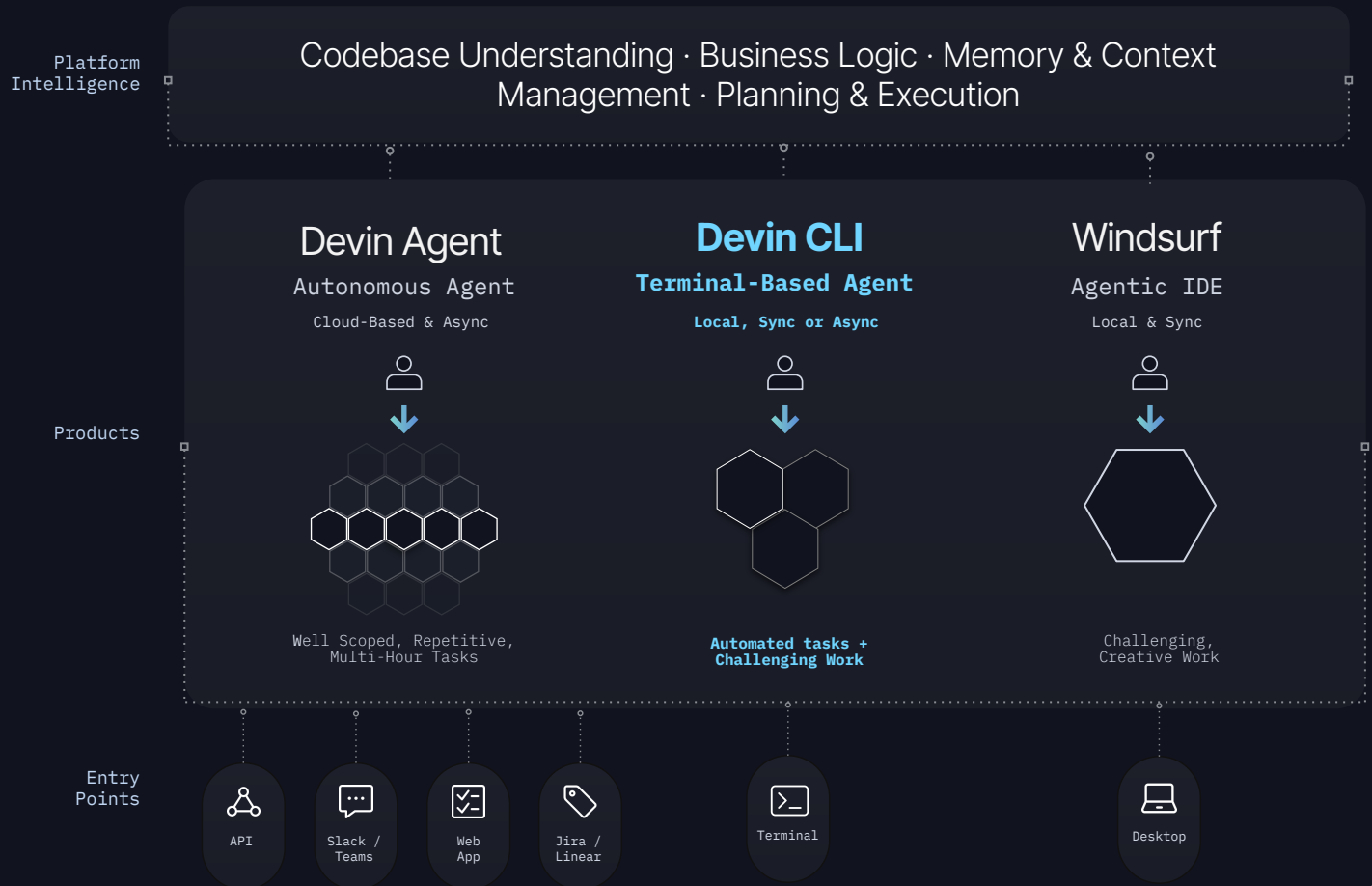
Windsurf 201

April 2026

We help companies deploy AI software engineers.



The Cognition Platform



Fast Context

find where the command palette is designed

I'll help you find where the command palette is designed in the VS Code codebase.

 Fast Context Find where the command palette is designed and implemented in VS Code

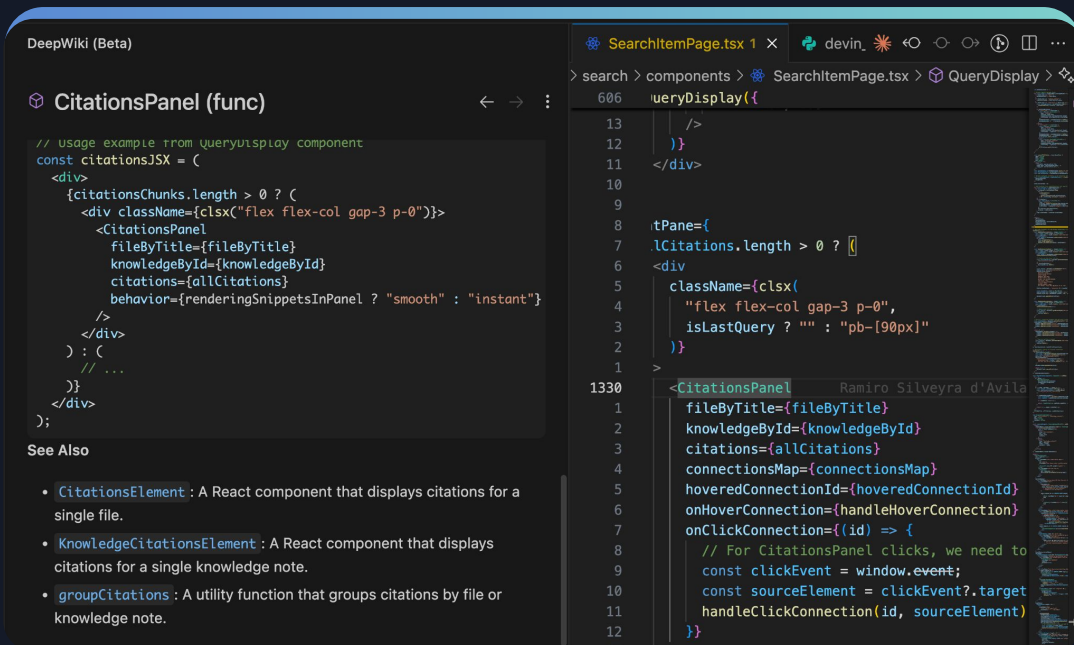
I need to understand the command palette implementation and design in this VS Code extension codebase. Let me start by exploring the codebase structure and searching for relevant references to "command palette" and related UI elements.

- ① Grepping command.*palette in /Users/roberthou/vscode-fork/vscode
- ① Grepping palette in /Users/roberthou/vscode-fork/vscode/src
- ② Grepping palette in /Users/roberthou/vscode-fork/vscode/extensions 1.2s
- ① Grepping command.*palette in /Users/roberthou/vscode-fork/vscode
- ① Grepping palette.*command in /Users/roberthou/vscode-fork/vscode/src
- ① Grepping palette in /Users/roberthou/vscode-fork/vscode
- ① Grepping Command.*Palette in /Users/roberthou/vscode-fork/vscode
- ① Grepping palette in /Users/roberthou/vscode-fork/vscode

Order-of-Magnitude Speed – SWE-grep retrieves context up to 20× faster than traditional search, with accuracy on par with frontier coding agents.

Noise-Free Precision – Returns only the most relevant file and line ranges, reducing context pollution and improving output quality.

Enterprise-Grade Scale – SWE-grep-mini peaks at ~2,800 tokens/sec, powering large-repo queries and keeping devs in flow.



DeepWiki for Windsurf

Instant Knowledge in Context - Hover + click to surface deep explanations of any function, class, or file directly inside Windsurf.

Accelerates Onboarding & Ramp - New devs understand codebases faster without endless handoffs or tribal knowledge hunts.

Bridges Code & Documentation - Push insights seamlessly into Cascade, keeping documentation fresh and aligned with live code.

Reduces Debug & Review Cycles - Shared understanding minimizes back-and-forth, errors, and rework across teams.

🔗 style (prop)

The `style` attribute in React allows you to apply CSS styles directly to JSX elements using JavaScript objects. This is a convenient way to dynamically style components without writing separate CSS classes.

Source Code

In React's type definitions, `style` is declared as an optional property that accepts `CSSProperties`:

```
TS index.d.ts#L2908
interface HTMLAttributes<T> {
  // ... other attributes ...
  style?: CSSProperties | undefined; // CSS styles as a JavaScript object
  // ... other attributes ...
}
```

This means that when you use the `style` prop in a JSX element, you should pass an object where the keys are camelCased CSS property names and the values are the corresponding CSS values.

Examples

Here's how you can use the `style` prop in a React component:

```
function MyComponent() {
  const buttonStyle = {
    backgroundColor: 'blue',
    color: 'white',
    padding: '10px 20px',
  };
}
```

Deep Codebase Understanding

4x More Accurate Retrieval – Proprietary approach outperforms embedding search while maintaining low latency.

DeepWiki Innovation – Automatically indexes repos, generating wikis with architecture diagrams, source links, and code summaries.

Seamless Access – Unified view of documentation, diagrams, and references across all repos.

The screenshot displays the Codemaps IDE interface. On the left, a sidebar shows the project structure. The main window is split into two panes. The left pane shows a 'Messaging Infrastructure Between Services' diagram, which is a complex flowchart illustrating the relationships between various services and their messaging components. The right pane shows the source code for the `EmployeeDTOConverter.java` file. The code is a Java class that implements the `EmployeeDTOConverter` interface, providing methods to convert domain models to DTOs and vice versa. The code is annotated with Codemaps, which link the diagram elements to the corresponding code blocks.

Codemaps

Messaging Infrastructure Between Services

Created December 23, 2025 at 12:48 PM

This codemap traces the complete messaging infrastructure that enables asynchronous communication between the monolith application and external systems. It covers three main messaging flows: invoice system integration via queue [16], payroll system integration with special message properties [2d], and notification broadcasting via topic [3d]. The map also shows the data transformation layer that converts domain models to DTOs [4a] and the underlying JMS configuration using ActiveMQ [5a].

```

1  /*
2  * Copyright © 2018 Colin But. All rights reserved.
3  */
4
5  package com.mycompany.entapp.snowman.infrastructure.messaging.converter;
6
7
8  import com.mycompany.entapp.snowman.domain.model.Employee;
9  import com.mycompany.entapp.snowman.domain.model.EmployeeProject;
10 import com.mycompany.entapp.snowman.domain.model.Project;
11 import com.mycompany.entapp.snowman.infrastructure.messaging.dto.EmployeeDTO;
12 import com.mycompany.entapp.snowman.infrastructure.messaging.dto.ProjectDTO;
13
14 import java.util.ArrayList;
15 import java.util.HashSet;
16 import java.util.Set;
17
18 public final class EmployeeDTOConverter {
19     private EmployeeDTOConverter() {
20     }
21
22     public static EmployeeDTO convertToEmployeeDTO(Employee employee) {
23         EmployeeDTO employeeDTO = new EmployeeDTO();
24         employeeDTO.setEmployeeId(employee.getId());
25         employeeDTO.setFirstName(employee.getFirstName());
26         employeeDTO.setSurname(employee.getSurname());
27         employeeDTO.setRole(employee.getRole());
28
29         Set<ProjectDTO> projectDTOSet = ProjectDTOConverter.convertToProject
30         4c Associates employee with their project DTOs -> next
31         employeeDTO.setProjectDTOList(new ArrayList<>(projectDTOSet));
32
33         return employeeDTO;
34     }
35
36     private static Set<Project> getProjectForEmployee(Employee employee) {
37         Set<Project> projects = new HashSet<>();
38         Set<EmployeeProject> employeeProjects = employee.getProjects();
39         for (EmployeeProject employeeProject : employeeProjects) {
40             projects.add(employeeProject.getProject());
41         }
42         return projects;
43     }
44 }

```

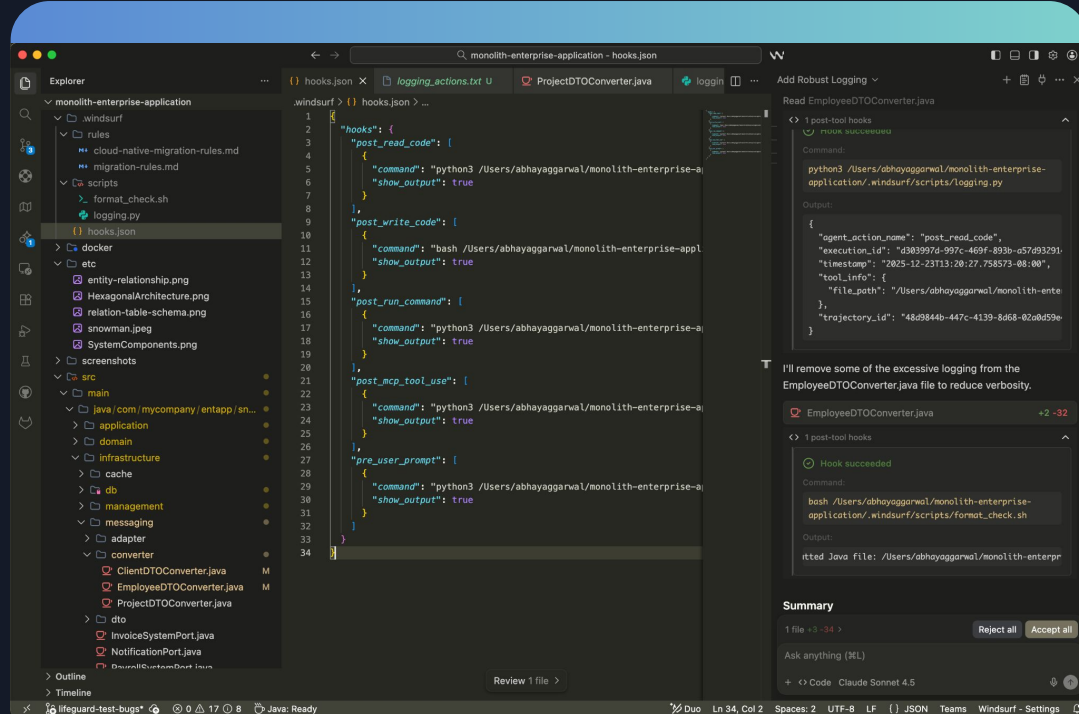
Codemaps

Code Intelligence Maps – Visualizes code execution order and relationships, turning complex repos into navigable hierarchies.

Collaborative Context – Shareable, browser-viewable maps align teams on structure and logic instantly.

AI-Ready Integration – Feeds contextual code insights directly into Cascade for smarter AI reasoning.

Hooks & Skills



Programmatic Controls - Hooks are a way for you to be able to execute custom commands automatically when Cascade is running

Full Coverage - Hooks cover events including every time a user prompts, Cascade reading files, Cascade writing to files, and more.

Logging - Hooks can be used to log all actions, including prompts, tool calls, and MCP usage

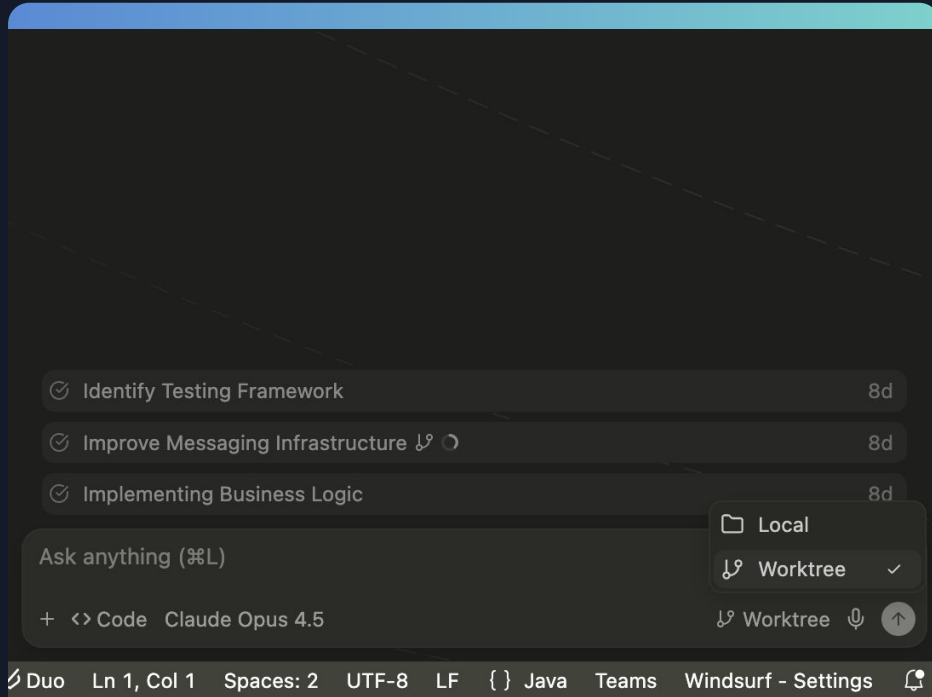
Guidance and Security - You can use hooks to programmatically apply standards to code written by Cascade, block certain files from being read, and more!

Git Worktrees

Parallelization: This is a tool for you to be able to spin up multiple Cascade sessions that can run on different worktrees.

Composability: The focus is for developers to be able to run and merge agentic changes that may have conflicts with each other otherwise.

Merging: After Cascade makes file changes in the worktree, you have the option of clicking “merge” to incorporate those changes back into your main workspace.



1 message queued

What is this app's framework

Enter to send queued message (↵)

+ <> Code Claude Sonnet 4.5



Queued Messages

Multiple Messages: If you already know what you are aiming to achieve in the next prompt, you can queue it within the same conversation

Conversation Management: This makes it a lot easier to manage multiple conversations at the same time

Adaptive Mode

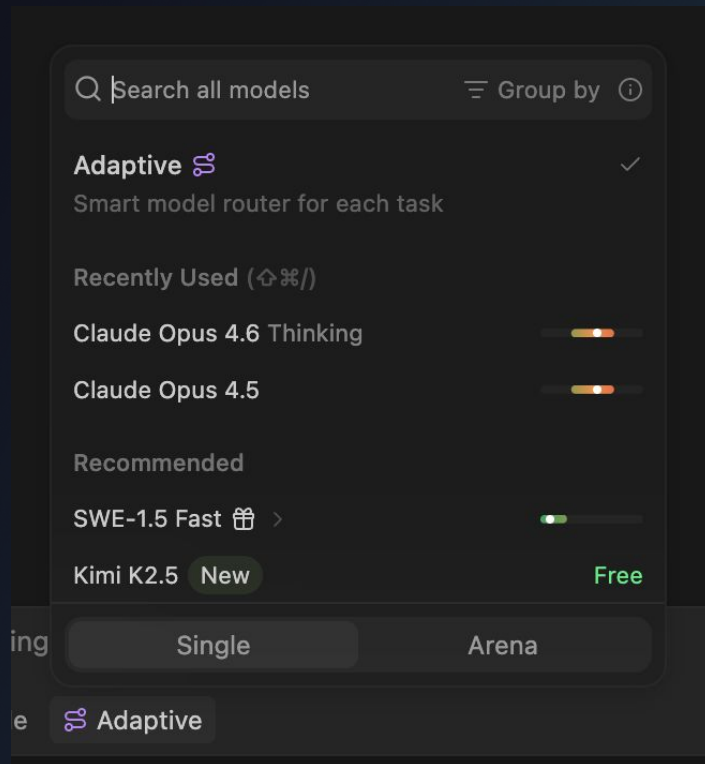
What it is: This is a new model mode that will route Cascade to use the optimal model for a given task.

Key features:

- Determines the correct model for a task
- Uses the existing model list for enterprise
- Adaptive usage of credits based on models
- Users can see the model used in the response statistics

What it enables:

You can reduce the cognitive load of switching between the right models by choosing adaptive mode.



Agent Command Center

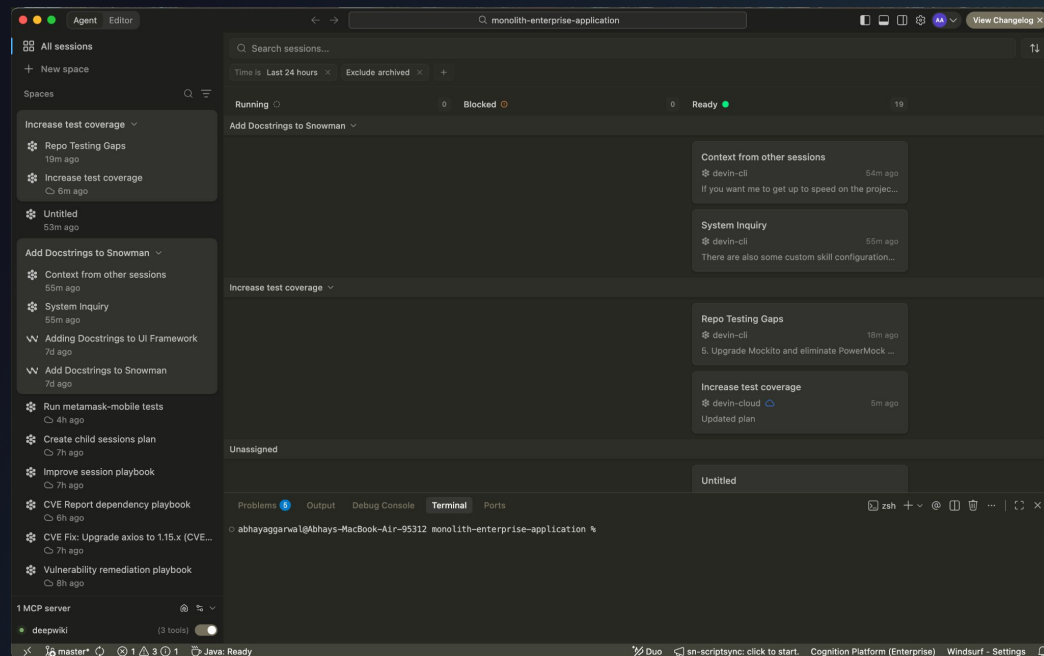
What it is: This is a new command center for you to be able to manage all of the agents that you are running on the Cognition Platform.

Key features:

- You can easily manage multiple agent "sessions" at once
- There is a kanban view to see which agents are active/blocked/ready for review
- An interface tailored to agent-first development

What it enables:

Developing and merging code faster while maintaining clear oversight of development



Spaces

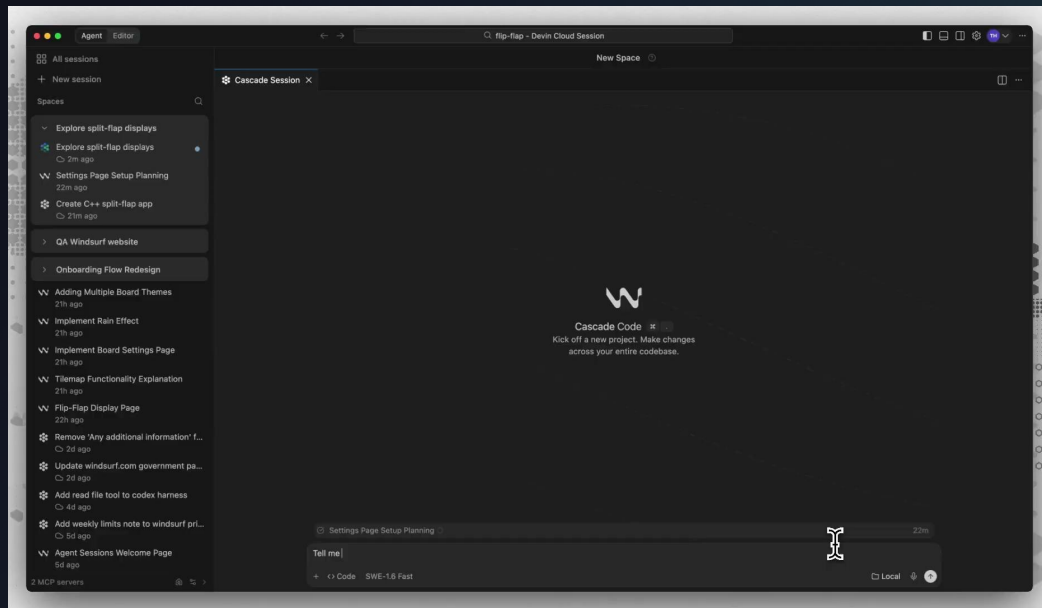
What it is: Spaces are a way to organize all of the work you do across multiple agents. All sessions within a space will share context with each other.

Key features:

- New sessions automatically share context within sessions
- Easily switch between tasks in different sessions
- Can use different types of agent within a space

What it enables:

Enable cross-agent work and better organization of agent context.



Support Engineering

- Windsurf has a **dedicated support engineering team in the USA & India**
- **Enterprise customers are prioritized by default**, make sure you are logged in when you file a ticket
- If you encounter any bugs with using Windsurf, **please first file a Support ticket, make sure to include your diagnostics log**
- **Please post in the Slack channel AFTER you have filed a ticket** with our Support team
- ***windsurf.com/support***

Need more help? Start a chat with support

